

Báo cáo thiết kế RTL CPU 4-bit

Ngày 26 tháng 7 năm 2026

Mục lục

1	Mục tiêu thiết kế	2
2	Cấu trúc source code	2
3	Giao diện top-level	2
4	Kiến trúc datapath	3
4.1	Các thanh ghi	3
4.2	Cập nhật tuần tự	3
5	Bộ điều khiển FSM	4
6	Tập lệnh	4
7	ALU 4-bit và flags	4
8	Reset và coding style	5
9	Môi trường kiểm thử RTL	5
10	Chạy mô phỏng và xem waveform	6
11	Kết quả synthesis tham chiếu	6
12	Giới hạn và hướng phát triển	6
13	Kết luận	7

1 Mục tiêu thiết kế

Thiết kế xây dựng một CPU đồng bộ có datapath 4-bit để minh họa đầy đủ chu trình thiết kế số từ RTL đến physical design. CPU thực thi instruction 8-bit, hỗ trợ các phép toán số học, logic, dịch bit, nhảy và dừng. RTL được viết bằng SystemVerilog và có thể tổng hợp bằng Yosys sang standard cell Sky130.

Các mục tiêu chính:

- datapath và địa chỉ rộng 4 bit;
- instruction rộng 8 bit;
- kiến trúc accumulator đơn giản;
- bộ điều khiển FSM ba trạng thái;
- reset bất đồng bộ active-low;
- instruction memory nằm ngoài CPU;
- RTL synthesizable, không suy diễn latch;
- có self-checking testbench và chương trình kiểm thử.

2 Cấu trúc source code

File	Chức năng
cpu_4bit.sv	Top-level CPU, các thanh ghi trạng thái và kết nối datapath.
control_unit.sv	Giải mã instruction và điều khiển FSM.
alu_4bit.sv	Thực hiện các phép toán số học/logic và tạo flags.
program_rom.sv	Mô hình ROM 16 từ dùng cho mô phỏng độc lập.
tb.sv	Self-checking RTL testbench.
memory/program.hex	Chương trình kiểm thử 16 byte.
filelist.f	Danh sách source dùng cho Xcelium.

3 Giao diện top-level

Module cpu_4bit có giao diện:

```
module cpu_4bit (  
    input logic clk,  
    input logic rst_n,  
    output logic [3:0] imem_addr,  
    input logic [7:0] imem_rdata,  
    output logic halted  
);
```

Signal	Width	Ý nghĩa
clk	1	Clock cạnh lên. Physical design và testbench cuối dùng chu kỳ 20 ns.
rst_n	1	Reset bất đồng bộ, tác động mức thấp.

Signal	Width	Ý nghĩa
imem_addr	4	Địa chỉ instruction, lấy trực tiếp từ program counter.
imem_rdata	8	Instruction đọc bất đồng bộ từ bộ nhớ ngoài.
halted	1	Bằng 1 khi CPU đã thực thi lệnh HLT.

Instruction memory không nằm trong core. Cách tách này giúp CPU có giao diện rõ ràng, tránh tổng hợp ROM test thành standard cells và cho phép thay đổi chương trình mà không sửa datapath.

4 Kiến trúc datapath

4.1 Các thanh ghi

CPU chứa các thanh ghi sau:

- pc[3:0]: program counter, tạo địa chỉ 0–15;
- ir[7:0]: instruction register;
- acc[3:0]: accumulator và toán hạng chính của ALU;
- zero_flag, carry_flag, borrow_flag, negative_flag, overflow_flag: các cờ trạng thái.

Instruction được chia thành:

$$\text{instruction}[7:0] = \underbrace{\text{opcode}[3:0]}_{\text{IR}[7:4]} \parallel \underbrace{\text{operand}[3:0]}_{\text{IR}[3:0]}.$$

Opcode chọn phép toán, còn operand có thể là immediate data hoặc jump target. Đây là encoding phù hợp với CPU nhỏ vì mỗi instruction chỉ cần một byte.

4.2 Cập nhật tuần tự

Các thanh ghi dùng nonblocking assignment:

```
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    pc <= 4'b0000;
    ir <= 8'b0000_0000;
    acc <= 4'b0000;
  end else begin
    if (ir_load) ir <= imem_rdata;
    if (pc_load) pc <= operand;
    else if (pc_inc) pc <= pc + 4'd1;
    if (acc_write) acc <= alu_result;
  end
end
```

Toán tử <= cập nhật thanh ghi ở NBA region ngay sau cạnh clock hiện tại; nó không tự tạo thêm một chu kỳ. Một chu kỳ pipeline chỉ xuất hiện khi dữ liệu phải đi qua hai thanh ghi liên tiếp hoặc khi FSM cố ý chia thao tác thành nhiều trạng thái.

5 Bộ điều khiển FSM

Control unit có ba trạng thái:

1. **FETCH**: nạp $IR \leftarrow imem_rdata$, tăng $PC \leftarrow PC + 1$, sau đó chuyển sang EXECUTE;
2. **EXECUTE**: giải mã opcode, phát control signals và thực thi instruction, thông thường quay lại FETCH;
3. **HALT**: giữ $halted=1$ và không thực thi thêm lệnh.

Luồng trạng thái cơ bản:

FETCH $\rightarrow$ EXECUTE $\rightarrow$ FETCH $\rightarrow \dots \rightarrow$ HALT.

Vì mỗi instruction dùng một chu kỳ FETCH và một chu kỳ EXECUTE, throughput cơ bản là một instruction trong hai chu kỳ. Đây là nguyên nhân `imem_addr` giữ nguyên qua hai clock phases; không phải do nonblocking assignment.

6 Tập lệnh

Opcode	Mnemonic	Hoạt động
0x0	NOP	Không thay đổi trạng thái datapath.
0x1	LDI	$ACC \leftarrow operand$.
0x2	ADD	$ACC \leftarrow ACC + operand$.
0x3	SUB	$ACC \leftarrow ACC - operand$.
0x4	AND	$ACC \leftarrow ACC \& operand$.
0x5	OR	$ACC \leftarrow ACC operand$.
0x6	XOR	$ACC \leftarrow ACC \wedge operand$.
0x7	NOT	$ACC \leftarrow \sim ACC$.
0x8	SHL	Dịch trái ACC một bit.
0x9	SHR	Dịch phải logic ACC một bit.
0xA	JMP	$PC \leftarrow operand$.
0xB	JZ	Nạp PC bằng operand nếu zero flag bằng 1.
0xC	INC	$ACC \leftarrow ACC + 1$.
0xD	DEC	$ACC \leftarrow ACC - 1$.
0xF	HLT	Chuyển FSM sang HALT.

Opcode 0xE chưa được định nghĩa; control unit xử lý opcode không hợp lệ giống NOP để thiết kế luôn có hành vi xác định.

7 ALU 4-bit và flags

ALU là logic tổ hợp, nhận hai toán hạng `a`, `b` cùng `alu_op`. Mọi output được gán giá trị mặc định trước câu lệnh `case`, do đó không suy diễn latch.

Flag	Ý nghĩa
zero	Result bằng 0.

Flag	Ý nghĩa
negative	Bằng bit MSB của result, biểu diễn dấu trong two's complement.
carry	Carry-out của ADD/INC hoặc bit bị dịch ra trong SHL/SHR.
borrow	Bằng 1 khi phép trừ unsigned có $a < b$; DEC tại 0 cũng tạo borrow.
overflow	Tràn signed của ADD, SUB, INC hoặc DEC.

Ví dụ carry và overflow của phép cộng được tính độc lập:

```
temp = {1'b0, a} + {1'b0, b};
result = temp[3:0];
carry = temp[4];
overflow = ~(a[3] ^ b[3]) & (result[3] ^ a[3]);
```

Carry mô tả tràn unsigned, trong khi overflow mô tả tràn signed.

8 Reset và coding style

Reset `rst_n` là asynchronous active-low:

```
always_ff @(posedge clk or negedge rst_n)
```

Khi reset, PC, IR, ACC, flags và FSM state đều về giá trị xác định. Logic tuần tự dùng `always_ff` và nonblocking assignment; logic tổ hợp dùng `always_comb` và blocking assignment. Yosys xác nhận không suy diễn latch cho ALU hoặc control unit.

9 Môi trường kiểm thử RTL

Testbench tạo clock 20 ns để khớp constraint physical design, giữ reset qua hai cạnh lên rồi nhả reset. Instruction memory được mô hình hóa như ROM bất đồng bộ zero-wait-state:

```
assign imem_rdata = program_memory[imem_addr];
```

Chương trình kiểm thử:

Addr	Byte	Instruction	ACC sau execute
0	13	LDI 3	3
1	25	ADD 5	8
2	32	SUB 2	6
3	80	SHL	12
4	90	SHR	6
5	F0	HLT	6

Testbench chờ `halted`, đợi thêm một cạnh clock rồi kiểm tra `dut.acc == 4'd6`. Nếu CPU không halt trước 500 ns, watchdog báo fatal error.

10 Chạy mô phỏng và xem waveform

Lệnh Xcelium:

```
xrun -64bit -sv -timescale 1ns/1ps -access +rwc -xmlbdirname runs/  
manual_tcl_util65/xcelium_rtl_20ns.d -f filelist.f -top cpu_4bit_tb -input  
xcelium_rtl_shm.tcl
```

Mở waveform:

```
simvision runs/manual_tcl_util65/rtl_wave.shm
```

Waveform đúng có chuỗi địa chỉ 0,1,2,3,4,5,6, dữ liệu 13,25,32,80,90,F0,00 và halted lên 1 sau khi HLT được execute. RTL và post-layout waveform có cùng trình tự chức năng. RTL đổi tín hiệu qua delta-cycle gần cạnh clock; post-layout có propagation delay do SDF.

11 Kết quả synthesis tham chiếu

Run Yosys tại runs/manual_pd_01/06-yosys-synthesis cho kết quả:

Chỉ tiêu	Kết quả
Top module	cpu_4bit
Số wire sau technology mapping	140
Số standard-cell instance	140
Số flip-flop dfrtp_2	19
Mapped cell area	1591.5264 μm^2

Mười chín flip-flop tương ứng với PC 4 bit, IR 8 bit, ACC 4 bit, năm flags và FSM state sau tối ưu/mapping. Synthesis tạo netlist standard-cell làm đầu vào cho floorplan và các bước physical design.

12 Giới hạn và hướng phát triển

- CPU chưa có data memory, stack hoặc register file.
- Mỗi instruction cần hai chu kỳ; có thể cải thiện bằng pipeline hoặc kiến trúc single-cycle.
- Instruction memory giả định asynchronous zero-wait-state; giao diện thực tế nên có handshake hoặc wait-state.
- JZ chỉ dùng zero flag; có thể bổ sung branch theo carry, negative và overflow.
- Cần testbench bổ sung để kiểm tra từng opcode, boundary values, signed overflow, carry và borrow.
- Có thể bổ sung assertions và functional coverage cho FSM, instruction decode và flags.

13 Kết luận

RTL CPU 4-bit có kiến trúc accumulator nhỏ gọn nhưng chứa đủ các thành phần cơ bản của một bộ xử lý: PC, IR, ALU, flags, control FSM và giao tiếp instruction memory. Coding style phân tách rõ logic tuần tự và tổ hợp, không suy diễn latch và tổng hợp thành công sang Sky130. Chương trình kiểm thử thực hiện đúng chuỗi lệnh và kết thúc với ACC bằng 6. Đây là nền tảng để thực hiện synthesis, physical design, signoff và post-layout gate-level simulation.